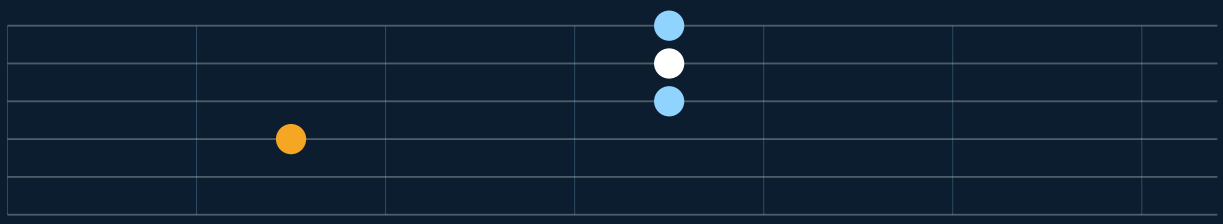
DROP ROUTE

Documento de Diseño Técnico del Software

Voicings de guitarra y rutas II–V–I
Base de desarrollo para Claude Code



Autor del proyecto: Jesús Quevedo Rodríguez

Revisión: base reconstruida y DR-001 · 13/10/2026
Repositorio: a01653/Drop_Route
Estado: juego pendiente de implementación

Índice

1 Objetivo y alcance 2

2 Arquitectura general 2

3 Tecnologías y dependencias 2

4 Estructura actual 3

5 Componentes previstos 4

6 Estado y persistencia 4

7 Modelo musical 5

8 Catálogo y construcción de voicings 5

9 Rutas y conducción de voces 5

10 Preguntas y corrección 6

11 Móvil y accesibilidad 6

12 Offline e instalación 6

13 Documentación y Claude 6

14 Flujo de datos 7

15 Pruebas y resultados reales 7

16 Construcción y publicación 7

17 Limitaciones actuales 8

18 Evolución 9

19 Apéndice: evidencia y límites 9

1 Objetivo y alcance

Drop Route servirá para memorizar voicings, inversiones y grados y practicar rutas II-V-I concretas. Ordenador y móvil. Primera etapa en C, con la apariencia de nuestras tarjetas.

Se preparan fuentes e instrucciones para que Claude Code implemente cada etapa mediante órdenes de trabajo. No necesita acceder al historial de esta conversación ni recibir las plantillas de nuevo.

2 Arquitectura general

Diseño previsto de aplicación cliente, sin backend.

Capa	Responsabilidad prevista
Datos	Posiciones y rutas procedentes de la plantilla.
Núcleo musical	Alturas físicas, grados, inversiones y solución de ruta.
Sesión	Pregunta, selección, evaluación y transición.
Presentación	Tarjeta SVG, selectores, fichas y resultados.
Persistencia	Configuración y progreso local con versión de esquema.

La geometría no decide la respuesta y los componentes no duplican el catálogo. Actualmente están implementados la extracción y el núcleo de auditoría de la base y, en `src/`, un esqueleto de cuatro capas:

Capa	Contenido actual
`src/data`	Tipos y acceso a `data/catalog.json` y `data/routes.json`, que siguen siendo la única definición de datos.
`src/music`	Altura física (MIDI) e inversión por bajo real. Funciones puras.
`src/session`	Solo la fase de arranque. Sin React.
`src/ui`	Pantalla de arranque con el estado de desarrollo.

La dependencia permitida es ui, session, music, data. ESLint rechaza que una capa importe otra superior y que data, music o session usen React, de modo que pueden probarse sin DOM. La persistencia no existe todavía, y tampoco el motor de preguntas ni los juegos.

3 Tecnologías y dependencias

Node.js 22.13 o superior (campo engines; la CI usa Node 24 según .nvmrc) ejecuta tests, auditorías y herramientas. Python 3.10 o superior extrae las constantes literales y genera el DTS. Lo localiza el lanzador `scripts/run-python.mjs`: prueba la variable PYTHON y después py -3, python y python3 en Windows, o python3 y python en otros sistemas. Si no hay una versión válida, el error lista lo probado y explica cómo resolverlo. Los tests y auditorías de la base con node:test no dependen de paquetes externos.

DR-001 fijó estas versiones en el lockfile de npm. Instalación: `npm ci --ignore-scripts`. No se copiaron dependencias innecesarias de Mástil.

Paquete	Versión	Nota
react, react-dom	19.3.0	Única dependencia de ejecución.
typescript	6.0.3	Se fija la serie 6.0 porque typescript-eslint 8.71.2 exige una versión inferior a 6.1. La 7.0.2 es posterior y no es compatible.
vite, @vitejs/plugin-react	8.3.4, 6.1.2	Build y servidor de desarrollo.
vitest	5.0.4	Tests unitarios y de capas, sin DOM.
eslint, @eslint/js	10.12.0, 10.0.1	Lint y límites entre capas.
typescript-eslint	8.71.2	Reglas para TypeScript.
eslint-plugin-react-hooks, eslint-plugin-react-refresh, globals	7.1.1, 0.5.7, 17.13.0	Reglas de React y entornos.
@playwright/test	1.64.0	E2E con Chromium.
@types/node, @types/react, @types/react-dom	24.19.2, 19.3.0, 19.3.0	Tipos.

Vite 8 y Vitest 5 exigen Node 22.12 y ESLint 10, 22.13; de ahí el mínimo de engines. Playwright necesita descargar el navegador con `npx playwright install chromium`; ese paso queda fuera de npm ci.

Pandoc y LuaLaTeX completo o XeLaTeX generan este documento. El estilo utiliza la paleta azul oscuro y familias tipográficas de la documentación de Mástil, con equivalentes si faltan TeX Gyre. Idioma español y sin partición de palabras al final de línea.

4 Estructura actual

Ruta	Contenido
`CLAUDE.md` / `AGENTS.md`	Reglas permanentes y flujo de trabajo.
`data/catalog.json`	Sets, fórmulas, afinación y 124 formas.
`data/routes.json`	Dieciséis rutas II-V-I completas.
references/	Tres fuentes originales y sus hashes.
scripts/	Extracción, auditoría, generación documental y lanzador de Python.
tests/	Pruebas con node:test, conservadas aparte: las de la base musical y las del utillaje.
`src/`	Capas `data`, `music`, `session` y `ui`, `main.tsx` y los tests Vitest junto a cada capa.
`e2e/`	Humo de Playwright sobre el build, en escritorio y móvil.
`index.html` y configuración	Vite, Vitest, Playwright, ESLint y dos `tsconfig` (aplicación y herramientas).

Ruta	Contenido
docs/	Requisitos, decisiones, tareas, validación y DTS.
`FILES_SHA256.json`	Instantánea histórica de los hashes de la base inicial del 13/10/2026, previa a DR-001. No es el estado actual ni se actualiza.
`RECOVERY_STATUS.json`	Instantánea histórica del estado de la reconstrucción, con sus cifras de entonces. No se actualiza.
.github/workflows/	Validación de la base y trabajo de aplicación, preparados para GitHub.

`npm run build` genera `dist/`, que Git ignora. El build solo incluye lo que importa `src/`: ni las referencias ni la plantilla HTML. No hay despliegue.

5 Componentes previstos

Componente	Responsabilidad
Configuración	Juego, Drop, set, dificultad y modo de ruta.
Tarjeta	Cuerdas, trastes, notas y estado neutro/solución.
Respuesta	Cuatro botones o dos desplegables.
Grados	Cuatro destinos, arrastre, dos toques y teclado.
Rutas	Partida fija y una candidata visible.
Corrección	Resultado por campos y avance explícito.

Ninguno de estos componentes existe todavía: la única pantalla actual es la de arranque, con el nombre, el estado de desarrollo y un resumen de los datos cargados. Su aspecto no anticipa el icono ni las tarjetas.

Compartir el renderizador conserva proporciones entre juegos. El icono final sigue pendiente; el brief está en `docs/VISUAL.md`.

6 Estado y persistencia

Modelo previsto: configuración, pregunta, pregunta verificada y regreso al menú. Cada pregunta identifica datos y guarda sus opciones y orden.

Seleccionar no evalúa. Verificar exige respuesta completa y produce un único resultado. Después se bloquea la respuesta. Siguiente genera otra pregunta y Salir vuelve al menú.

La corrección no sobrescribe lo que contestó el usuario. Si falla el V de una ruta, el paso V-I debe partir del V correcto.

Hoy la sesión solo tiene la fase de arranque; el modelo anterior es el diseño que llegará con los juegos. DR-006 añadirá progreso local con esquema independiente de la versión de app. Ninguna persistencia está implementada todavía.

7 Modelo musical

Afinación estándar EADGBE. MIDI al aire, cuerdas 6 a 1: 40,45,50,55,59,64. Altura real igual a MIDI al aire más traste.

Calidades evaluables en C: maj7 (0,4,7,11), 7 (0,4,7,10), m7 (0,3,7,10) y m7(b5) (0,3,6,10). Grados reales, incluidas b3, b5 y b7.

Inversión por bajo real: raíz, tercera, quinta o séptima. No deducirla del primer elemento de un array.

Cdim7 se conserva como referencia: 0,3,6,9 y bb7. La fuente rotula su séptima como °7. El acorde es simétrico y no se evalúa sin fijar raíz y contexto.

8 Catálogo y construcción de voicings

No se generan posiciones nuevas. Drop 2 tiene tres sets; Drop 3 y Drop 2+4, dos cada uno. Cuatro calidades y cuatro inversiones dan 112 posiciones evaluables. Hay otras doce disminuidas de referencia.

La auditoría verifica cuerda/traste, fórmula, grados, bajo y orden por Drop. La extracción analiza datos literales sin ejecutar JavaScript. --check confirma que los JSON coinciden con la fuente sin sobrescribir.

Mantener ventana corta de las tarjetas. Algunas formas abarcan cinco trastes: no recortar para imponer una ventana de tres.

9 Rutas y conducción de voces

Normalizadas solo Drop 2 sets 1 y 2, mayor y menor, cuatro inversiones iniciales: dieciséis rutas.

II	V	I
Fundamental	2ª	Fundamental
1ª	3ª	1ª
2ª	Fundamental	2ª
3ª	1ª	3ª

Mayor Dm7-G7-Cmaj7. Menor Dm7(b5)-G7-Cm7. Comparar con la ruta exacta del material. Otra inversión de G7 puede ser musicalmente válida y no corresponder a este ejercicio.

No aplicar un optimizador que sustituya estas rutas ni afirmar mínimo universal. C6 aparece como sustitución opcional en ciertos pasos de la fuente; no se convierte automáticamente en otro acorde evaluable.

10 Preguntas y corrección

Diseño acordado: fácil elige nombre con cuatro botones; medio añade inversión en un segundo desplegable; difícil añade colocar grados sobre círculos.

Opciones únicas y una correcta. En tests, fixture o semilla reproducible. El banco difícil es fijo con alteraciones y distractores; ofrecer solo los grados correctos daría una pista.

Corrección por acorde, inversión y cada destino. Acierto completo exige todos los campos del nivel. Registrar una sola respuesta por primer intento y bloquear cambios al verificar.

11 Móvil y accesibilidad

El fondo es el tramo corto de mástil de las tarjetas, no un mástil completo de doce trastes. En rutas, partida arriba, candidata debajo y selector 1/2/3/4; no cuatro candidatas pequeñas simultáneas.

Pregunta con círculos grises y fondo común. No revelar solución mediante nombre, título, tooltip o nombre accesible de las opciones. Al corregir, recuperar colores musicales y usar texto/símbolos además de rojo/verde.

Arrastre con tolerancia, alternativa de tocar ficha y destino y operación por teclado. Áreas táctiles mayores que los círculos visibles. Se prevén E2E a 360,390 y 430 px; aún no ejecutables.

12 Offline e instalación

DR-006 prevé PWA instalable, caché de recursos propios y recarga sin conexión tras una primera carga completa.

Progreso por dispositivo, sin sincronización, cuentas, backend o notificaciones. Probar almacenamiento bloqueado, datos dañados y migración. No hay manifest PWA, service worker o instalación comprobada todavía.

13 Documentación y Claude

`CLAUDE.md` importa `AGENTS.md` y dirige al contrato y a la orden solicitada. Las reglas permanentes están versionadas. Las tareas concretas se describen en DR-001 a DR-007 y se ejecutan por separado.

Se adapta el flujo de Mástil Escalas: rama por objetivo, pruebas dirigidas durante ajustes, revisión funcional y batería final aplicable. Un cambio material de diseño actualiza el DTS principal y sus derivados en la misma rama.

La preparación se conserva como paquete duradero. El 13/10/2026 el usuario autorizó la publicación de la base completa en el repositorio público, incluidos los originales.

14 Flujo de datos

Previsto: configuración selecciona registros; generador crea pregunta estable; tarjeta dibuja geometría; el usuario compone respuesta; evaluador calcula resultado según contrato.

En rutas, pregunta identifica ruta y paso. Cambiar candidata no cambia la partida. El estado de selección no altera los datos musicales.

La sesión y el núcleo musical se prueban sin DOM; los componentes consumen el catálogo sin duplicar posiciones. ESLint hace cumplir esta separación y Vitest se ejecuta en el entorno `node`, sin DOM.

15 Pruebas y resultados reales

La reconstrucción ejecutó 24 tests con node:test: todos pasan, sin omitidos. Verifican catálogo completo, rutas, hashes de fuentes y rechazo de errores deliberados. DR-001 los conserva sin cambios en `tests/music-catalog.test.mjs` y añade en `tests/tooling.test.mjs` 11 tests de utillaje: el lanzador de Python y que el workflow y la configuración de Playwright conservan sus pasos. Con ellos node:test ejecuta 35. `npm test` ejecuta primero node:test y después Vitest, de modo que ninguno queda fuera. Las dos instantáneas históricas de la base inicial, descritas en la estructura del proyecto, conservan sus cifras de entonces y ningún test las comprueba.

La auditoría pasa con 124 posiciones (112 evaluables y doce de referencia) y 16 rutas. Hay oráculos independientes para D2S1 mayor primera inversión y D2S2 menor fundamental.

Se prueba también una forma G7 válida en otra inversión que no cumple la ruta. Los tests no se limitan a comparar las tablas consigo mismas.

DR-001 añade 15 tests Vitest en cinco ficheros y 10 ejecuciones E2E (cinco escenarios en dos proyectos). Vitest comprueba las capas sin DOM: las 124 posiciones y los 48 pasos de ruta del catálogo devuelven por bajo real la inversión que declaran, el bajo desplazado un semitono se detecta y el orden del array no influye. También comprueba el resumen de datos y la pantalla de arranque, y que la configuración amplía las exclusiones predeterminadas con e2e, tests y los worktrees. Playwright prueba el build servido por vite preview en un proyecto de escritorio y otro móvil: título, estado, datos empaquetados, que dist/ se generó durante esa ejecución y que el preview sirve exactamente su index.html, ausencia de errores de consola y de red, y falta de desplazamiento horizontal a 360 px.

El lint y las reglas de capas se probaron con importaciones prohibidas deliberadas, y las exclusiones de Vitest con tests que fallarían si se ejecutaran. Esas pruebas fueron manuales y no forman parte de la suite. Los resultados concretos están en el registro de validación de DR-001 del repositorio. No son cobertura del juego, que todavía no existe.

16 Construcción y publicación

npm run docs:build regenera LaTeX/PDF desde el Markdown. Guarda hashes del Markdown, derivados, plantilla y generador para detectar desactualizaciones.

La CI preparada tiene dos trabajos con `npm ci --ignore-scripts`. El de la base usa Node 24 y Python 3.10, el mínimo admitido, y ejecuta data:check, audit:music, test:base y docs:check. El de la aplicación ejecuta typecheck, lint, test:unit, build, la descarga de Chromium con sus dependencias del sistema y test:e2e. Un test de node:test comprueba que el workflow conserva

esos pasos. Ninguno se ha ejecutado en GitHub: esta rama no se ha subido. El código en Git tampoco equivale a tener el juego desplegado.

Comando	Función
`npm run dev`	Servidor de desarrollo de Vite.
`npm run build`	Comprueba tipos y genera `dist/`.
`npm run preview`	Sirve `dist/`.
`npm run typecheck`	`tsc` sobre aplicación y herramientas.
`npm run lint`	ESLint, con los límites entre capas.
`npm test`	node:test de la base y después Vitest.
`npm run test:e2e`	Playwright sobre un build recién generado.

Playwright genera el build y lo sirve con vite preview al ejecutar los E2E. Nunca reutiliza un servidor existente: con el puerto 4173 ocupado, la ejecución falla antes de probar nada. El humo comprueba además que dist/index.html es posterior al inicio de la ejecución y que el preview sirve ese mismo fichero, de modo que no se prueba una compilación anterior ni el servidor de desarrollo. Ningún preview queda iniciado. Los scripts `test:unit` y `test:base` ejecutan cada corredor por separado.

El comando docs:build funciona con Pandoc 3.12.1 y LuaLaTeX. El script y la plantilla se adaptaron en DR-001. LaTeX falla con las rutas temporales cortas de Windows, que contienen una tilde. Pandoc 3.12 envuelve las tablas con un contador none que la plantilla no definía, y ya no emite hypertarget en los títulos. Además, `.gitattributes` fija ahora LF para los `.tex`, porque en Windows la copia con CRLF cambiaba los hashes del registro.

El primer push fue bloqueado por revisión automática debido al repositorio público y los materiales originales. El 13/10/2026 se obtuvo autorización explícita del usuario para publicar la base completa. Las futuras publicaciones siguen requiriendo una orden que las cubra.

17 Limitaciones actuales

- La interfaz se reduce a la pantalla de arranque: no hay juegos, tarjetas, icono final, progreso, PWA ni revisión funcional móvil de los juegos. El humo móvil usa emulación de Chromium, no un dispositivo real.
- Playwright solo prueba Chromium; Firefox y WebKit no están configurados.
- Python se ejecutó en esta revisión solo con la versión 3.13.7. El mínimo 3.10 se comprobó por gramática y por las API usadas, y se ejercitará en la CI al publicar.
- Solo se normalizaron las rutas Drop 2 sets 1 y 2 del HTML.
- Los PDF tienen 21 páginas de imágenes cada uno; no se han transcrito íntegramente.
- Cdim7 y C6 no son calidades evaluables de la primera etapa.
- Transposición y audio quedan para otra etapa.
- La reconstrucción conserva contratos y datos, no recupera el commit local perdido byte a byte.
- Hosting y licencia del código nuevo pendientes; las referencias conservan su autoría.

18 Evolución

DR-001 configuró las herramientas y queda entregada para revisión, sin fusionar; DR-002 crea tarjetas e icono; DR-003 y DR-004, reconocimiento; DR-005, rutas; DR-006, progreso/PWA; DR-007, cierre documental y pruebas.

Convertir lo planificado en descripción de implementación solo después de inspeccionar y probar el código. No extender una tarea a mejoras no pedidas.

19 Apéndice: evidencia y límites

Verificados: mismas tres referencias, SHA-256, extracción, notas físicas, fórmulas, grados, inversiones, rutas y 24 tests ejecutados nuevamente. Documentación reconstruida desde las decisiones visibles de la conversación.

El juego no está implementado. Las comprobaciones de DR-001 se registran en el directorio de validación del repositorio, junto con la compilación del DTS anterior, su revisión visual y la validación documental de la base.